Introduction to Data Science

Programming I: Functions and debugging

Simon Munzert
Hertie School | GRAD-C11/E1339

Table of contents

Recap: tidyverse basics

What is the tidyverse?

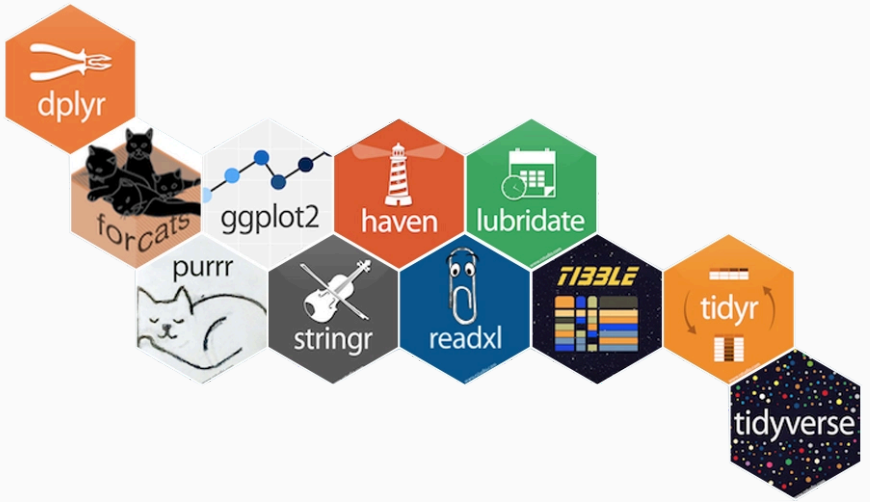
R packages for data science

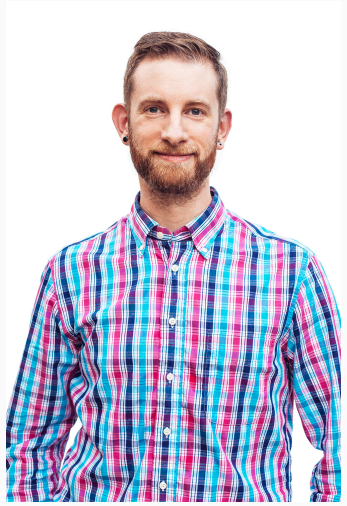
- Let's take it from the tidyverse website:

"The tidyverse is an opinionated collection of R packages designed for data science. All packages share an underlying design philosophy, grammar, and data structures."

- It's the contribution of many people of the R community.
- Hadley Wickham had a key role in shaping it by developing many of the core packages, such as `ggplot2`, `dplyr`, `tidyr`, `tibble`, and `stringr`.
- Install the complete tidyverse with:

```
R> install.packages("tidyverse")
```



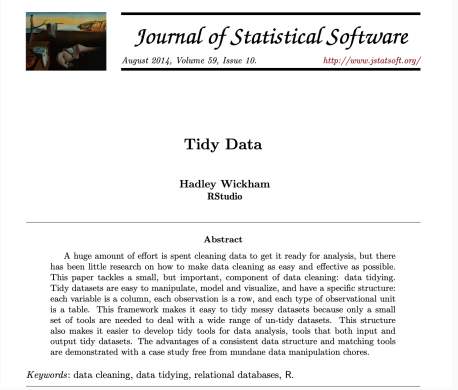


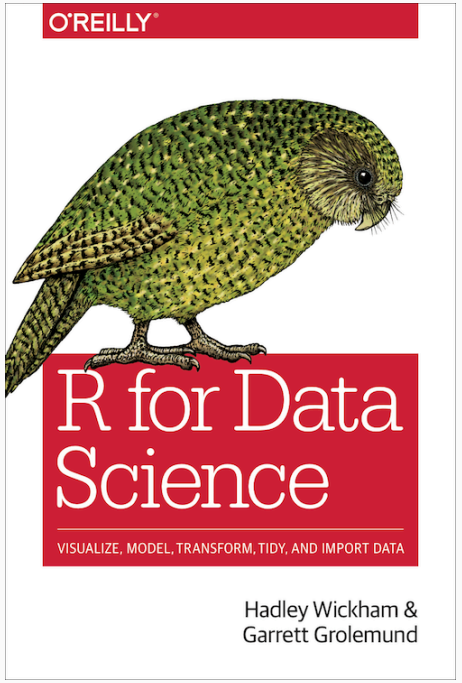
Hadley Wickham

A guide to the tidyverse

Valuable resources

- Welcome to the Tidyverse, a quick overview from many tidyverse contributors
- Tidy data, a foundational paper on data wrangling and structuring, by Hadley Wickham, 2014, *Journal of Statistical Software*; check here for a hands-on vignette based on the `tidyr` package
- The tidyverse design guide, a (soon-to-be book) manifesto to promote design consistency across the tidyverse
- R for Data Science, our main textbook for this course





Tidyverse packages

Loading the tidyverse

```
R> library(tidyverse)
```

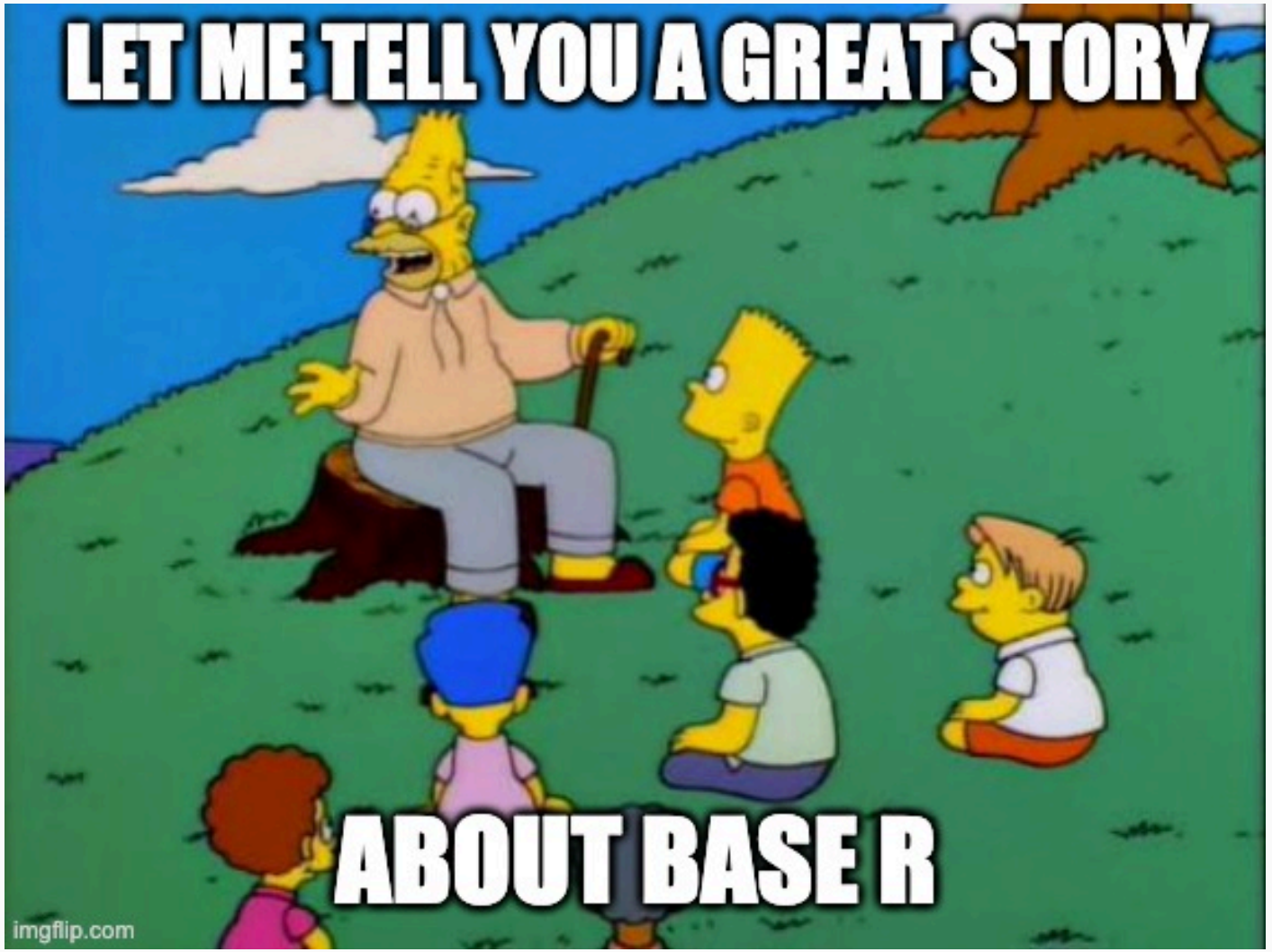
- In addition to the currently 8 core packages, the tidyverse includes many others for more specialized usage.[1]
- We'll use several of these additional packages during the remainder of this course (e.g., the `rvest` package for web scraping). Bear in mind that these packages will have to be loaded separately.
- See here for an overview, or just in R directly:

```
R> tidyverse_packages()
```

```
 [1] "broom"        "conflicted"   "cli"          "dbplyr"       "dplyr"        "dtplyr"       "forcats"
 [8] "ggplot2"      "googledrive"  "googlesheets4" "haven"        "hms"          "httr"         "jsonlite"
[15] "lubridate"    "magrittr"     "modelr"       "pillar"       "purrr"        "ragg"         "readr"
[22] "readxl"       "reprex"       "rlang"        "rstudioapi"   "rvest"        "stringr"      "tibble"
[29] "tidyr"        "xml2"         "tidyverse"
```

[1] It also includes a *lot* of dependencies upon installation. This is a matter of some controversy.

Tidyverse vs. base R



Tidyverse vs. base R: what's the difference?

- Both are compatible. You can wrangle your data with `dplyr`, plot it with `ggplot2`, and model it with yet another package.
- Ultimately, the tidyverse is just a bunch of (hugely popular!) packages that share design principles.
- Often, tidyverse packages don't reinvent the wheel. Instead, they offer more consistency in naming, arguments, and output (among other things).
- For instance, compare function naming principles (`tidyverse::snake_case` vs `base::period.case` rule; more on these conventions later) in these examples:

tidyverse	base
`?readr::read_csv`	`?utils::read.csv`
`?dplyr::if_else`	`?base::ifelse`
`?tibble::tibble`	`?base::data.frame`

- If you call up the above examples, you'll see that the tidyverse alternative typically offers some enhancements or other useful options (and sometimes restrictions) over its base counterpart.
- And **remember:** There are (almost) always multiple ways to achieve a single goal in R.

Tidyverse vs. base R: what's the difference? *cont.*

Tidyverse

`Credit` sawiki.com

Base R

`Credit` multimedialab.be

Tidyverse vs. base R: what to use?

Stories from the past

- When I started to learn R ~19 years ago, there was no tidyverse. The learning curve felt much steeper. I often switched back to Stata for data wrangling.
- As the tidyverse grew, R became more convenient to use for the entire research pipeline.
- There's simply no need for you to live through the same pain.

Why we start with the tidyverse

- Because clever people think it's the right way.
- Documentation + community support are great.
- Having a consistent syntax makes it easier to learn.

You still will want to check out base R alternatives later

- Base R is extremely flexible and powerful (and stable).
- There are some things that you'll have to venture outside of the tidyverse for.
- A combination of tidyverse and base R is often the best solution to a problem.
- Excellent base R data manipulation tutorial here.

Functions

Tidy programming basics

"Tidy programming" is not a strictly defined practice in the tidyverse. However, there are some common programming strategies that help you keep your code and workflow tidy. These include:

- Pipes (you already learned how to use them ✅)
- User-generated functions
- Functional programming with `purrr`

The latter two are extremely helpful - in particular when you are confronted with iterative tasks.

We will now learn the basics of creating your own functions and functional programming with R. There is much more to learn about these topics, so we will revisit them as the course progresses.

Functional programming

R is a functional programming (FP) language. As Hadley Wickham puts it in Advanced R:

> This means that it provides many tools for the creation and manipulation of functions. In particular, R has what's known as first-class functions. You can do anything with functions that you can do with vectors: you can assign them to variables, store them in lists, pass them as arguments to other functions, create them inside functions, and even return them as the result of a function.

R encourages you to use and build your own functions to solve problems. Often, this implies decomposing a large problem into small pieces, and solving each of them with independent functions.

There is much more to learn about functions and functional programming. Useful resources include:

- The chapter on functions in R for Data Science.
- The section on functional programming in Advanced R.
- The R packages book. In a way, bundling functions in a package is sometimes the next logical step.

Creating functions

Why creating functions?

That's a legit question. There are 24,000+ **packages** on CRAN (and many, many more on GitHub and other repositories) containing zillions of functions. Why should you create yet another one?

- Every data science project is unique. There are problems only you have to solve.
- For problems that are repetitive, you'll quickly look for options to automate the task.
- Functions are a great way to automate.

Examples where creating functions makes sense

1. You want to scrape thousands of websites. This implies multiple steps, from downloading to parsing and cleaning. All these steps can be achieved with existing functions, but the fine-tuning is specific to the set of websites. You build one (or a set of) scraping functions that take the websites as input and return a cleaned data frame ready to be analyzed.

2. You want to estimate not one but multiple models on your dataset. The models vary both in terms of data input and specification. Again, based on existing modeling functions you tailor your own, allowing you to run all these models automatically and to parse the results into one clean data frame.

What kinds of functions will you write?

Most functions you will write as a data scientist fall into one of three types:

	Vector functions	Data frame functions	Plot functions
Input	One or more vectors	A data frame (+ column names)	A data frame (+ column names)
Output	A vector	A data frame	A plot
Typical use	Inside `mutate()`, `filter()`, or `summarize()`	As a custom verb wrapping a repeated dplyr pipeline	As a recipe wrapping repeated ggplot2 code
Example	`z_score(x)`	`grouped_mean(df, ...)`	`histogram(df, var)`

Some foreshadowing - there's more out there, but no reason to panic yet:

- **Functions made for iteration**, which you pass on to other functions such as `purrr::map()`. See next session!
- **Functions with side effects**, which you call for what they do (saving a file, printing a message), not for what they return.
- **Bundles of functions**, aka packages. Once your functions prove useful beyond a single project, bundling them in a package is the next logical step.

Basic syntax

Writing your own function in R is easy with the `function()` function[1]. The basic syntax is as follows:

```
R> my_func <- function(ARGUMENTS) {
+     OPERATIONS
+     return(VALUE)
+   }
```

[1] Yes, a function to create functions. 🤯

Basic syntax

Writing your own function in R is easy with the `function()` function[1]. The basic syntax is as follows:

```
R> my_func <- function(ARGUMENTS) {
+     OPERATIONS
+     return(VALUE)
+   }
```

- We write functions to apply them later. So, we have to give them a name. Here, we name it "`my_func`".
- Also, our function (almost) always needs input, plus we want to specify how exactly the function should behave. We can use arguments for this, which are specified as arguments of the `function()` function.

[1] Yes, a function to create functions. 🤯

Basic syntax

Writing your own function in R is easy with the `function()` function[1]. The basic syntax is as follows:

```
R> my_func <- function(ARGUMENTS) {
+     OPERATIONS
+     return(VALUE)
+   }
```

- Next, we specify anything we want the function to do.
- This comes in between curly brackets, `{...}`.
- Importantly, we can recycle arguments by calling them by their name.

[1] Yes, a function to create functions. 🤯

Basic syntax

Writing your own function in R is easy with the `function()` function[1]. The basic syntax is as follows:

```
R> my_func <- function(ARGUMENTS) {
+     OPERATIONS
+     return(VALUE)
+   }
```

- Finally, we specify what the function should return.
- This could be a list, data.frame, vector, sentence - or anything else really.
- Note that R automatically returns the final object that is written (not: assigned!) in your function by default. Still, my recommendation is that you get into the habit of assigning the return object(s) explicitly with `return()`.

[1] Yes, a function to create functions. 🤯

Basic syntax

Writing your own function in R is easy with the `function()` function[1]. The basic syntax is as follows:

```
R> my_func <- function(ARGUMENTS) {
+     OPERATIONS
+     return(VALUE)
+   }
```

- Oh, and don't forget to close the curly brackets...

[1] Yes, a function to create functions. 🤯

Basic syntax

Writing your own function in R is easy with the `function()` function[1]. The basic syntax is as follows:

```
R> my_func <- function(ARGUMENTS) {
+     OPERATIONS
+     return(VALUE)
+   }
```

Let's try it out with a simple example function - one that converts temperatures from Fahrenheit to Celsius:[2]

```
R> fahrenheit_to_celsius <- function(temp_F) {
+   temp_C <- (temp_F - 32) * (5/9)
+   return(temp_C)
+ }
```

[1] Yes, a function to create functions. 🤯

[2] Courtesy of Software Carpentry.

Basic syntax

Writing your own function in R is easy with the `function()` function[1]. The basic syntax is as follows:

```
R> my_func <- function(ARGUMENTS) {
+     OPERATIONS
+     return(VALUE)
+   }
```

Let's try it out with a simple example function - one that converts temperatures from Fahrenheit to Celsius:[2]

```
R> fahrenheit_to_celsius <- function(temp_F) {
+   temp_C <- (temp_F - 32) * (5/9)
+   return(temp_C)
+ }
```

- Our function has an intuitive name.
- Also, it takes just one thing as input, which we call `temp_F`.

[1] Yes, a function to create functions. 🤯

[2] Courtesy of Software Carpentry.

Basic syntax

Writing your own function in R is easy with the `function()` function[1]. The basic syntax is as follows:

```
R> my_func <- function(ARGUMENTS) {
+     OPERATIONS
+     return(VALUE)
+   }
```

Let's try it out with a simple example function - one that converts temperatures from Fahrenheit to Celsius:[2]

```
R> fahrenheit_to_celsius <- function(temp_F) {
+   temp_C <- (temp_F - 32) * (5/9)
+   return(temp_C)
+ }
```

- We now take up the argument `temp_F`, do something with it, and store the output in a new object, `temp_C`.
- Importantly, that object only lives within the function. When the function is run, we cannot access it from the environment.

[1] Yes, a function to create functions. 🤯

[2] Courtesy of Software Carpentry.

Basic syntax

Writing your own function in R is easy with the `function()` function[1]. The basic syntax is as follows:

```
R> my_func <- function(ARGUMENTS) {
+     OPERATIONS
+     return(VALUE)
+   }
```

Let's try it out with a simple example function - one that converts temperatures from Fahrenheit to Celsius:[2]

```
R> fahrenheit_to_celsius <- function(temp_F) {
+   temp_C <- (temp_F - 32) * (5/9)
+   return(temp_C)
+ }
```

- Finally, the output is returned.

[1] Yes, a function to create functions. 🤯

[2] Courtesy of Software Carpentry.

Basic syntax

Writing your own function in R is easy with the `function()` function[1]. The basic syntax is as follows:

```
R> my_func <- function(ARGUMENTS) {
+     OPERATIONS
+     return(VALUE)
+   }
```

Let's try it out with a simple example function - one that converts temperatures from Fahrenheit to Celsius:

```
R> fahrenheit_to_celsius <- function(temp_F) {
+   temp_C <- (temp_F - 32) * (5/9)
+   return(temp_C)
+ }
```

Now, let's try out the function:

```
R> fahrenheit_to_celsius(451)
```

```
  [1] 232.7778
```

Pretty hot, isn't it?

[1] Yes, a function to create functions. 🤯

Functions: default argument values, if(), else()

Let's make the function a bit more complex, but also more fun.

```
R> temp_convert <-
+   function(temp, from = "f") {
+   if (!(from %in% c("f", "c"))){
+     stop("No valid input
+           temperature specified.")
+   }
+   if (from == "f") {
+     out <- (temp - 32) * (5/9)
+   } else {
+     out <- temp * (9/5) + 32
+   }
+   if((from == "c" & temp > 30) |
+      (from == "f" & out > 30)) {
+     message("That's damn hot!")
+   }else{
+     message("That's not so hot.")
+   }
+   return(out) # return temperature
+ }
```

Functions: default argument values, if(), else()

Let's make the function a bit more complex, but also more fun.

- By giving `from` a default value (`"f"`), we ensure that the function returns valid output when only the key input, `temp`, is provided.

```
R> temp_convert <-
+   function(temp, from = "f") {
+   if (!(from %in% c("f", "c"))){
+     stop("No valid input
+           temperature specified.")
+   }
+   if (from == "f") {
+     out <- (temp - 32) * (5/9)
+   } else {
+     out <- temp * (9/5) + 32
+   }
+   if((from == "c" & temp > 30) |
+      (from == "f" & out > 30)) {
+     message("That's damn hot!")
+   }else{
+     message("That's not so hot.")
+   }
+   return(out) # return temperature
+ }
```

Functions: default argument values, if(), else()

Let's make the function a bit more complex, but also more fun.

- By giving `from` a default value (`"f"`), we ensure that the function returns valid output when only the key input, `temp`, is provided.
- `if() {...}` allows us to make conditional statements. Here, we test for the validity of the input for argument `from`.

```
R> temp_convert <-
+   function(temp, from = "f") {
+   if (!(from %in% c("f", "c"))){
+     stop("No valid input
+           temperature specified.")
+   }
+   if (from == "f") {
+     out <- (temp - 32) * (5/9)
+   } else {
+     out <- temp * (9/5) + 32
+   }
+   if((from == "c" & temp > 30) |
+      (from == "f" & out > 30)) {
+     message("That's damn hot!")
+   }else{
+     message("That's not so hot.")
+   }
+   return(out) # return temperature
+ }
```

Functions: default argument values, if(), else()

Let's make the function a bit more complex, but also more fun.

- By giving `from` a default value (`"f"`), we ensure that the function returns valid output when only the key input, `temp`, is provided.
- `if() {...}` allows us to make conditional statements. Here, we test for the validity of the input for argument `from`.
- If the condition is not met, the function breaks and prints a message.

```
R> temp_convert <-
+   function(temp, from = "f") {
+   if (!(from %in% c("f", "c"))){
+     stop("No valid input
+           temperature specified.")
+   }
+   if (from == "f") {
+     out <- (temp - 32) * (5/9)
+   } else {
+     out <- temp * (9/5) + 32
+   }
+   if((from == "c" & temp > 30) |
+      (from == "f" & out > 30)) {
+     message("That's damn hot!")
+   }else{
+     message("That's not so hot.")
+   }
+   return(out) # return temperature
+ }
```

Functions: default argument values, if(), else()

Let's make the function a bit more complex, but also more fun.

- By giving `from` a default value (`"f"`), we ensure that the function returns valid output when only the key input, `temp`, is provided.
- `if() {...}` allows us to make conditional statements. Here, we test for the validity of the input for argument `from`.
- If the condition is not met, the function breaks and prints a message.
- With `else()`, we specify what to do if the `if()` condition is not met.

```
R> temp_convert <-
+   function(temp, from = "f") {
+   if (!(from %in% c("f", "c"))){
+     stop("No valid input
+           temperature specified.")
+   }
+   if (from == "f") {
+     out <- (temp - 32) * (5/9)
+   } else {
+     out <- temp * (9/5) + 32
+   }
+   if((from == "c" & temp > 30) |
+      (from == "f" & out > 30)) {
+     message("That's damn hot!")
+   }else{
+     message("That's not so hot.")
+   }
+   return(out) # return temperature
+ }
```

Functions: default argument values, if(), else()

Let's make the function a bit more complex, but also more fun.

- By giving `from` a default value (`"f"`), we ensure that the function returns valid output when only the key input, `temp`, is provided.
- `if() {...}` allows us to make conditional statements. Here, we test for the validity of the input for argument `from`.
- If the condition is not met, the function breaks and prints a message.
- With `else()`, we specify what to do if the `if()` condition is not met.
- Make R more talkative with `message()`. Future-You will like it!

```
R> temp_convert <-
+   function(temp, from = "f") {
+   if (!(from %in% c("f", "c"))){
+     stop("No valid input
+           temperature specified.")
+   }
+   if (from == "f") {
+     out <- (temp - 32) * (5/9)
+   } else {
+     out <- temp * (9/5) + 32
+   }
+   if((from == "c" & temp > 30) |
+      (from == "f" & out > 30)) {
+     message("That's damn hot!")
+   }else{
+     message("That's not so hot.")
+   }
+   return(out) # return temperature
+ }
```

Anonymous functions

In R, functions are objects in their own right. They aren't automatically bound to a name. If you choose not to give the function a name, you get an **anonymous function**. You use an anonymous function when it's not worth the effort to give it a name.

Examples:

```
R> map(char_vec, function(x) paste(x, collapse = "|"))
R> integrate(function(x) sin(x) ^ 2, 0, pi)
```

Anonymous functions

In R, functions are objects in their own right. They aren't automatically bound to a name. If you choose not to give the function a name, you get an **anonymous function**. You use an anonymous function when it's not worth the effort to give it a name.

As of `R 4.1.0`, there's a new shorthand syntax for anonymous functions: `\(x)`.

Example:

```
R> (function (x) {paste(x, 'is awesome!')})('Data science') # old syntax
```

```
  [1] "Data science is awesome!"
```

```
R> (\(x) {paste(x, 'is awesome!')})('Data science') # new syntax
```

```
  [1] "Data science is awesome!"
```

Anonymous functions

In R, functions are objects in their own right. They aren't automatically bound to a name. If you choose not to give the function a name, you get an **anonymous function**. You use an anonymous function when it's not worth the effort to give it a name.

As of `R 4.1.0`, there's a new shorthand syntax for anonymous functions: `\(x)`. This plays along nicely with the (native) pipe when we want to pass content to the RHS but not to the first argument.

Anonymous functions

In R, functions are objects in their own right. They aren't automatically bound to a name. If you choose not to give the function a name, you get an **anonymous function**. You use an anonymous function when it's not worth the effort to give it a name.

As of `R 4.1.0`, there's a new shorthand syntax for anonymous functions: `\(x)`. This plays along nicely with the (native) pipe when we want to pass content to the RHS but not to the first argument.

Example:

```
R> mtcars |> subset(cyl == 4) |> (\(x) lm(mpg ~ disp, data = x))()
```

`...` (Dot-dot-dot)

Functions can have a special argument `...` (pronounced *dot-dot-dot*). In other programming languages, this type of argument is often called varargs (short for variable arguments), or ellipsis. With it, a function can take any number of additional arguments. That is potentially very powerful!

A common application is to use `...` to pass those additional arguments on to another function.

Toy example:

```
R> my_list_generator <- function(y, z) {
+   list(y = y, z = z)
+ }
R> my_list_generator_2 <- function(x, ...) {
+   my_list_generator(...)
+ }
R>
R> str(my_list_generator_2(x = 1, y = 2, z = 3))
```

```
  List of 2
   $ y: num 2
   $ z: num 3
```

Real-life example:

```
R> map(.x, .f, ...)
R> map(mtcars, mean, na.rm = TRUE)
```

Arguments:

- `.x`: A list or atomic vector
- `.f`: A function
- `...`: Additional arguments passed on to the mapped function.

Data frame functions

The problem

Once you catch yourself repeating the same dplyr pipeline over and over, the obvious next step is to wrap it in a function - a **data frame function** that works like a custom dplyr verb.

Say we often compute group-wise means. Easy to functionalize, right?

- We pass the data frame plus the names of a grouping variable and a target variable.
- The function pipes them through `group_by()` and `summarize()`.
- What could possibly go wrong?

```
R> grouped_mean <-
+   function(df, group_var, mean_var) {
+   df |>
+     group_by(group_var) |>
+     summarize(
+       mean = mean(mean_var,
+                   na.rm = TRUE))
+ }
```

```
R> grouped_mean(flights, origin, arr_delay)
```

```
#> Error in `group_by()`:
#> ! Must group by variables found in `.data`.
#> ✖ Column `group_var` is not found.
```

😱 dplyr complains that there is no column called `group_var` - even though we clearly meant "the variable *stored in* `group_var`", i.e. `origin`. Welcome to the problem of **indirection**.

Indirection and tidy evaluation

The solution

R's behavior is a feature, not a bug. dplyr verbs use **tidy evaluation**: your code is evaluated *inside the data frame*, which is why you can refer to columns as if they were ordinary objects (no `flights$origin` needed).

However: `group_by(group_var)` is taken literally. dplyr looks for a column named `group_var` and cannot know that we mean the column whose name is stored in it.

The fix is called **embracing**: Wrapping an argument in doubled curly braces, `{{ }}`, tells dplyr to look down the tunnel 👀 (inside the argument) and use its contents.

Embrace whenever you pass column names on to a dplyr verb. This one trick covers most data frame functions you'll ever write.

```
R> grouped_mean <-
+   function(df, group_var, mean_var) {
+   df |>
+     group_by({{ group_var }}) |>
+     summarize(
+       mean = mean({{ mean_var }},
+                   na.rm = TRUE))
+ }
R> grouped_mean(flights, origin, arr_delay)
```

```
  # A tibble: 3 × 2
    origin  mean
    <chr>  <dbl>
  1 EWR     9.11
  2 JFK     5.55
  3 LGA     5.78
```

Data frame functions: a real-life example

Once you've internalized `{{ }}`, you can build yourself little helpers for everyday data work. Here is `summary6()`, which computes six summaries of any numeric variable at once:[1]

```
R> summary6 <- function(data, var) {
+   data |> summarize(
+     min = min({{ var }}, na.rm = TRUE),
+     mean = mean({{ var }}, na.rm = TRUE),
+     median = median({{ var }}, na.rm = TRUE),
+     max = max({{ var }}, na.rm = TRUE),
+     n = n(),
+     n_miss = sum(is.na({{ var }})),
+     .groups = "drop"
+   )
+ }
```

[1] Straight from R for Data Science, Ch. 25.

```
R> diamonds |> summary6(carat)
```

```
# A tibble: 1 × 6
    min  mean median   max     n n_miss
  <dbl> <dbl>  <dbl> <dbl> <int>  <int>
1   0.2 0.798    0.7  5.01 53940      0
```

Because `summarize()` does all the heavy lifting, our function inherits its superpowers - it works on grouped data, too:

```
R> diamonds |> group_by(cut) |> summary6(carat)
```

```
# A tibble: 5 × 7
  cut         min  mean median   max     n n_miss
  <ord>     <dbl> <dbl>  <dbl> <dbl> <int>  <int>
1 Fair       0.22 1.05     1     5.01  1610      0
2 Good       0.23 0.849    0.82  3.01  4906      0
3 Very Good  0.2  0.806    0.71  4     12082      0
4 Premium    0.2  0.892    0.86  4.01 13791      0
5 Ideal      0.2  0.703    0.54  3.5  21551      0
```

Data-masking vs. tidy-selection

Tidy evaluation comes in two flavors. Knowing which one you're dealing with helps you write functions - and read documentation:

Data-masking

- Used in functions that **compute with** variables: `filter()`, `mutate()`, `summarize()`, `arrange()`, `group_by()`, ...
- Arguments are expressions involving columns:

```
R> flights |> filter(arr_delay > 60)
R> flights |> mutate(speed = distance / air_time)
```

Tidy-selection

- Used in functions that **select** variables: `select()`, `relocate()`, `rename()`, `across()`, ...
- Arguments are column positions, ranges, or helpers:

```
R> flights |> select(year:day)
R> flights |> relocate(starts_with("arr"))
```

How do you know which flavor a given argument uses? Check the help file: the argument descriptions of, e.g., `?dplyr::filter` vs. `?dplyr::select` explicitly say `<data-masking>` or `<tidy-select>`. The good news: embracing with `{{ }}` works for both.

Data-masking vs. tidy-selection *cont.*

What if you want to use **tidy-selection inside a data-masking function** - say, group by an arbitrary *set* of variables?

Meet `pick()`, which lets you smuggle tidy-selection into data-masking arguments:

- `pick({{ group_vars }})` selects any number of columns...
- ... and hands them over to `group_by()`.

There's more where this came from (`across()`, injection with `!!`, the walrus operator `:=`, ...). No need to master these now - it's enough to know that the programming vignettes exist for when you meet them in the wild.

```
R> count_missing <-
+   function(df, group_vars, x_var) {
+   df |>
+     group_by(pick({{ group_vars }})) |>
+     summarize(
+       n_miss = sum(is.na({{ x_var }})),
+       .groups = "drop")
+ }
R> flights |>
+   count_missing(c(year, month, day),
+                 dep_time) |>
+   print(n = 4)
```

```
  # A tibble: 365 × 4
     year month   day n_miss
    <int> <int> <int>  <int>
  1  2013     1     1      4
  2  2013     1     2      8
  3  2013     1     3     10
  4  2013     1     4      6
  # ℹ 361 more rows
```

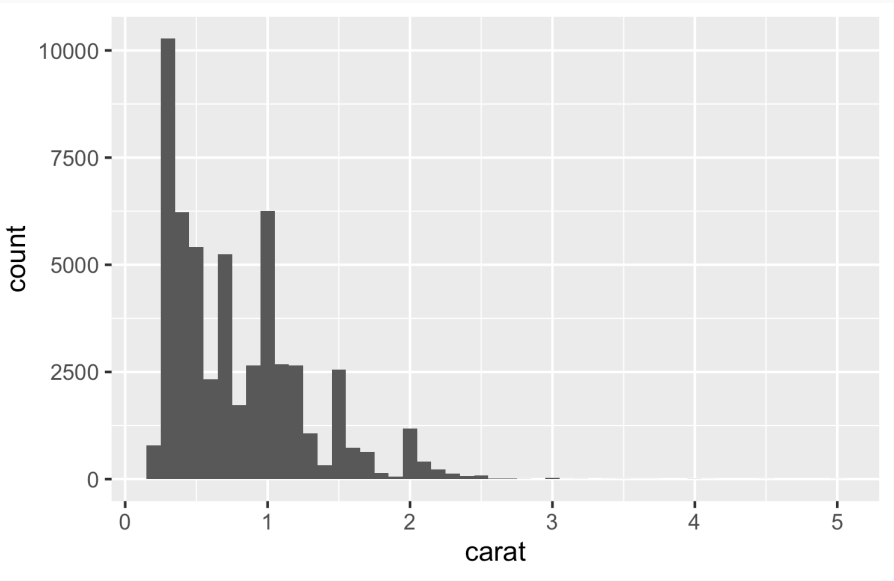
Plot functions

The same logic carries over to ggplot2, because `aes()` is a data-masking function, too. So if you're tired of copy-pasting the same plotting code, write a **plot function** and embrace its arguments:

```
R> histogram <-
+   function(df, var, binwidth = NULL) {
+   df |>
+     ggplot(aes(x = {{ var }})) +
+     geom_histogram(binwidth = binwidth)
+ }
```

- The function takes a data frame and a variable and returns a ggplot object.
- That means you can still add layers with `+` as usual (see on the right).

```
R> diamonds |> histogram(carat, 0.1)
```




```
R> diamonds |>
+   histogram(carat, 0.1) +
+   labs(x = "Size (in carats)")
```

📦 Interactive exercise: Will it run?

No coding needed - just vote! Each snippet below runs in a fresh R session with `tidyverse` and `nycflights13` loaded. For each one: does it return a result (🟢) or throw an error (🔴)? Vote by show of hands - and be ready to defend your prediction!

Snippet 1

```
R> mean_delay <- function(df) {
+   df |>
+     summarize(m = mean(dep_delay, na.rm = TRUE))
+ }
R> mean_delay(flights)
```

Snippet 2

```
R> mean_of <- function(df, var) {
+   df |>
+     summarize(m = mean(var, na.rm = TRUE))
+ }
R> mean_of(flights, dep_delay)
```

Snippet 3

```
R> mean_of <- function(df, var) {
+   df |>
+     summarize(m = mean({{ var }}, na.rm = TRUE))
+ }
R> mean_of(flights, dep_delay)
```

Snippet 4

```
R> dep_delay <- 10
R> flights |>
+   summarize(m = mean(dep_delay, na.rm = TRUE))
```

📦 Interactive exercise: Will it run? (solutions)

Snippet 1: 🟢 runs. The column name `dep_delay` is hard-coded, and data masking finds it inside `df`. Boring, but a useful baseline - indirection only becomes an issue when column names travel through arguments.

Snippet 2: 🔴 errors. `object 'dep_delay' not found`. There is no column called `var` in `flights`, so dplyr falls back to the environment - where `var` refers to `dep_delay`, which doesn't exist outside the data frame.

Snippet 3: 🟢 runs. Embracing `{{ var }}` tunnels the column name into `summarize()`. This is Snippet 2, fixed.

Snippet 4: 🟢 runs - but what does it return? The mean of the *column* `dep_delay` (about 12.6 minutes), not `10`. When data and environment offer the same name, the data mask wins. A great feature - until it silently hides a typo. 🙃

Take-home message: When wrapping dplyr or ggplot2 code in a function, ask yourself where each name is looked up - in the data or in the environment. If a column name enters through an argument, embrace it: `{{ }}`.

Writing functions with AI assistance

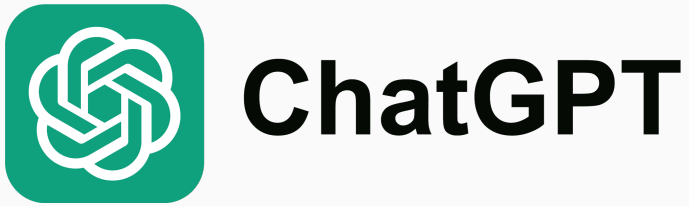
Not every function you plan to write is unique, nor is every problem you want to solve functionally.

Claude, ChatGPT, GitHub Copilot and other AI-based coding tools can help you a lot in finding functional solutions you can describe but not verbalize (yet).

Later in your coding life, I encourage you to use AI for this purpose, but be aware of the necessity to (a) debug and (b) assign credit where due.

Let's try it out with one of the following prompts:

- *Write an R function that capitalizes the first letter of each word in a character vector.*
- *Write an R function that allows me to play one round of black jack.*



Project management

Taming chaos

In the data science workflow, there are two sorts of **surprises** and cognitive stress:

1. Analytical (often good)
2. Infrastructural (almost always bad)

Analytical surprise is when you learn something from or about the data.

Infrastructural surprise is when you discover that:

- You can't find what you did before.
- The analysis code breaks.
- The report doesn't compile.
- The collaborator can't run your code.

Good project management lets you focus on the right kind of stress.

Keeping Future-you happy

- It’s often tempting to set up a project assuming that you will be the only person working on it, e.g. as homework.
- That’s almost never true.
- Coauthors and collaborators happen to the best of us.
- Even if not, there's someone else who you always have to keep happy: Future-you.
- Future-you is really the one you organize your projects for.
- They are who you use version control for (see later).
- Most importantly, they are who will enjoy the fruits of your data science labor, or have to fight back your chaos.
- So, be kind to Future-you. Establish a good workflow. You'll thank yourself later.



Project setup

You should **always** think in terms of projects.

A project is a **self-contained unit of data science work** that can be

- Shared
- Recreated by others
- Packaged
- Dumped

A project contains

- Content, e.g., raw data, processed data, scripts, functions, documents and other output
- Metadata, e.g., information about tools for running it (required libraries, compilers), version history

For R projects

- Projects are folders/directories.
- Metadata is the RStudio project (`.Rproj`) files (perhaps augmented with the output of renv for dependency management) and `.git`.

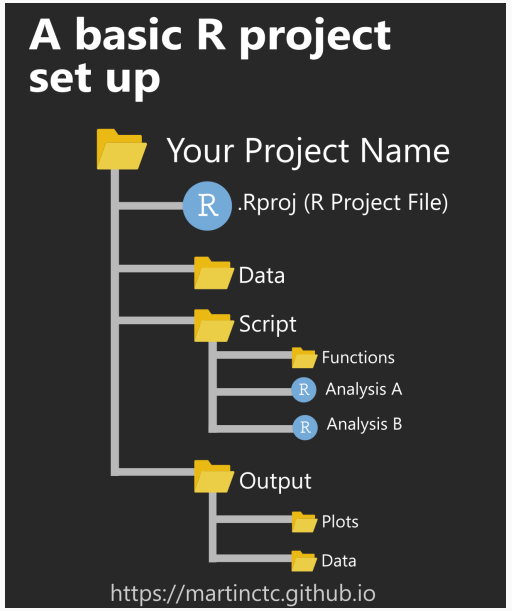
Setup: the folder structure

Structuring your working directory

- One folder contains everything inside it.
- Directories keep things separate that should be separated.
- You decide on the fundamental structure. The project decides on the details.

Further thoughts

- Ideally, your project folder can be relocated without problem.
- Keep input separate from output. Definitely separate raw from processed data!
- Structure should be capable of evolution. More data, cases, models, output formats shouldn't be a problem.




```
.
└── my_awesome_project
    ├── src
    ├── output
    ├── data
    │   ├── raw
    │   └── processed
    ├── README.md
    ├── run_analyses.R
    └── .gitignore
```

`Credit` Chris/r-bloggers.com

Setup: the paths

Good paths

- All internal paths are relative.
- They are invariant to moving/sharing the project.
- Examples:
  - `"preprocessing.R"`
  - `"figures/model-1.png"`
  - `"../data/survey.RDa"`

Bad paths

- Using `setwd()` is bad practice 99% of the times.
- Absolute paths are bad paths. Don't feed functions with paths like `"/Users/me/data/thing.sav"`.
- Those paths will not work outside your computer (or maybe not even there, some days/weeks/months ahead).

The working directory

- Set it manually once per session (do `Session > Set Working Directory > Choose Directory`). Then all your good paths will "just work".
- Better yet, get it right automatically by opening RStudio with clicking on the script you want to work with. This will set the location of the script as working directory (which should be your working assumption, too).
- Even better yet, have the metadata set it for you:
  - Open your session by opening (choosing, clicking on) `myproject.Rproj`
  - Then you'll get the path set for you.
- That's probably better than the previous option because you might not want your `code` directory to be the working directory.

Setup: the code structure

Naming scripts

- Files should have short, descriptive names that indicate their purpose.
- I recommend the use of telling verbs.
- Names should only include letters and numbers with dashes `-` or underscores `_` to separate words.
- Use numbering to indicate the order in which files should be run:
  - `0-setup.R`
  - `1-import-data.R`
  - `2-preprocess-data.R`
  - `3-describe-uptake.R`
  - `4-analyze-uptake.R`
  - `5-analyze-experiment.R`

Modularizing scripts

- Write short, modular scripts. Every script serves a purpose in your pipeline.
- This makes things easier to debug.
- At the beginning of a script you might want to document input and output.

Setup: the code structure (cont.)

Talk to Future-you

- Describe your code, e.g. by starting with a description of what it does. If you comment/describe a lot, consider using an R Markdown (`.Rmd`) file instead of a simple `.R` script.
- Put the setup first (e.g., `library()` and `source()`).
- You might want to outsource the loading of packages to a separate script that is imported in the first step (`source("functions.R")`) or just declared the first script in the pipeline.
- Always comment more than you usually do.

Structuring your code

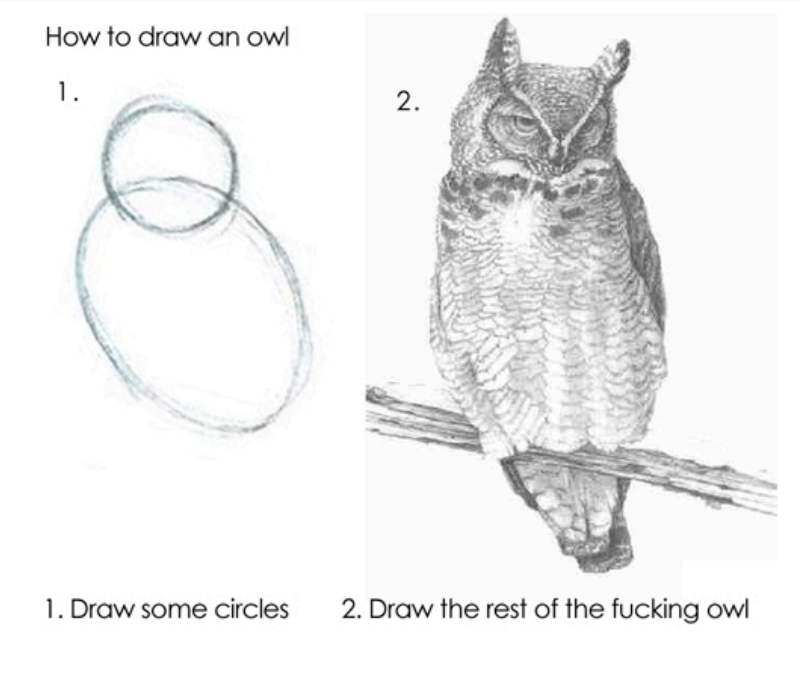
- Even with modularized code, scripts can become long. Structure helps to keep an overview.
- Use commented lines as section/subsection heads.
- RStudio creates a "table of contents" when you name your code chunks as follows (`#` followed by title and `---`):

```
R> # Import data --------------
R>
R> dat <- read_csv("dat.csv")
```

Setup: the rest

More things to consider

- There'd be more to say on how to establish a good project workflow, including how to
  - store/organize raw and derived data,
  - deal with output in form of graphs and tables,
  - link everything together from start (project setup) to finish (knitting the report)
  - separate coding for the record and experimental coding.
- But there's limited value in teaching you all that upfront.
- The truth is: You'll likely refine your own workflow over time. I just saved you some initial pain (hopefully).
- Do check out other people's experiences and opinions, e.g., here or here.



Managing your R project in two simple steps

Coding style

Coding style: the basics

Why adhere to a particular style of coding?

- It reduces the number of arbitrary decisions you have to consciously make during coding. We make an arbitrary decision (convention) once, not always ad hoc.
- It provides consistency.
- It makes code easier to write.
- It makes code easier to read, especially in the long term (i.e. two days after you've closed a script).

What are questions of style?

- Questions of style are a matter of opinion.
- We will mostly follow Hadley Wickham’s opinion as expressed in the "tidyverse style guide".
- We'll consider how to
  - name,
  - comment,
  - structure, and
  - write.

Naming things

Surprisingly many things can go wrong with naming

"There are only two hard things in Computer Science: cache invalidation and naming things." - *Phil Karlton*

`Credit` karlton.org

`Credit` Mashable

Naming files

- Code file names should be meaningful and end in `.R`.
- Avoid using special characters in file names. Stick with numbers, letters, dashes (`-`), and underscores (`_`).
- Some examples:

```
# Good
fit_models.R
utility_functions.R

# Bad
fit models.R
foo.r
stuff.r
```

- If files should be run in a particular order, prefix them with numbers:

```
00_download.R
01_explore.R
...
09_model.R
10_visualize.R
```

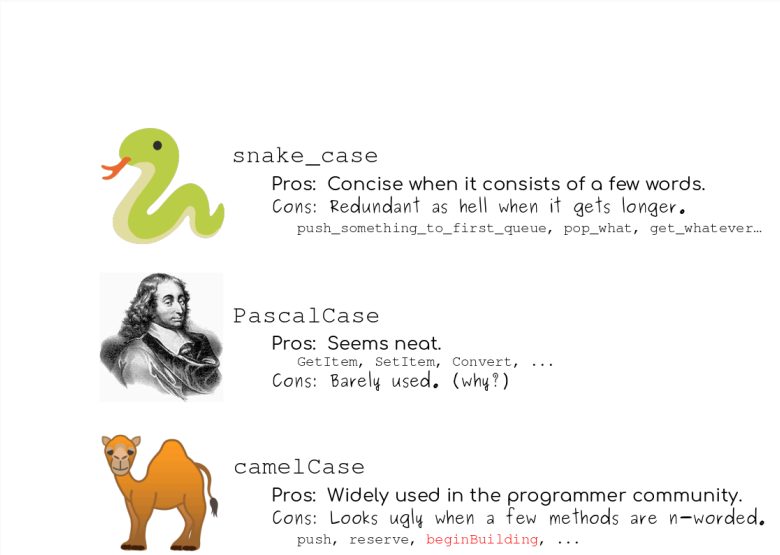
Naming objects and variables

- There are various conventions of how to write phrases without spaces or punctuation. Some of these have been adapted in programming, such as camelCase, PascalCase, or snake_case.
- The `tidyverse` way: Object and variable names should use only lowercase letters, numbers, and underscores.
- Examples:

```
# Good
day_one # snake_case
day_1 # snake_case

# Less good
dayOne # camelCase
DayOne # PascalCase
day.one # dot.case

# Dysfunctional
day-one # kebab-case
```



`Credit` cassert24/Reddit

Naming functions

- In addition to following the general advice for object names, strive to use verbs for function names:

```
# Good
add_row()
permute()

# Bad
row_adder()
permutation()
```

- Also, try avoiding function names that already exist, in particular those that come with a loaded package.
- This often implies a trade-off between shortness and uniqueness. In any case, you would try to avoid situations that force you disambiguate functions with the same name (as in `dplyr::select`; see "R packages").
- Check out this Wikipedia page) or this Stackoverflow post for more background on naming conventions in programming!
- For more good advice on how to name stuff, see this legendary presentation by Jenny Bryan.

Commenting on things

Why comment at all?

- It’s often tempting to set up a project assuming that you will be the only person working on it, e.g. as homework. But that's almost never true.
- You have project partners, co-authors, principals.
- Even if not, there's someone else who you always have to keep happy: Future-you.
- Comment often to make Future-you happy about Past-you by document what Present-You is doing/thinking/planning to do.

Past-you

Present-you

Future-you

Commenting on things *cont.*

General advice

- Each line of a comment should begin with the comment symbol and a single space: `#`
- Use comments to record important findings and analysis decisions.
- If you need comments to explain what your code is doing, consider rewriting your code to be clearer.
- But: comments can work well as "sub-headlines".
- If you discover that you have more comments than code, consider switching to R Markdown.
- (Longer) comments generally work better if they get their own line.

```
R> # define job status
R> dat$at_work <- dat$job %in% c(2, 3)
R> dat$at_work <- dat$job %in% c(2, 3) # define job
```

Giving structure

- Use commented lines together with dashes to break up your file into easily readable chunks.
- RStudio automatically detects these chunks and turns them into sections in the script outline.

```
R> # Input/output --------------------
R>
R> # input
R> c("data/survey2021.csv")
R>
R> #  output
R> c("survey_2021_cleaned.RData",
+   "resp_ids.csv")
R>
R> # Load data ----------------------
R>
R> # Plot data ----------------------
```

Other stuff

- Use **spaces** generously, but not too generously. Always put a space after a comma, never before, just like in regular English.
- Use `<-`, not `=`, for **assignment**.
- For **logical operators**, prefer `TRUE` and `FALSE` over `T` and `F`.
- To facilitate readability, **keep your lines short**. Strive to limit your code to about 80 characters per line.
- If a **function call is too long** to fit on a single line, use one line each for the function name, each argument, and the closing bracket.
- Use **pipes**. When you use them, they should always have a space before it, and should usually be followed by a new line.

Spacing

```
R> # Good
R> mean(x, na.rm = TRUE)
R> height <- (feet * 12) + inches
R>
R> # Bad
R> mean(x,na.rm=TRUE)
R> mean ( x, na.rm = TRUE )
R> height<-feet*12+inches
```

Piping

```
R> babynames %>%
+   filter(name %>% equals("Kim")) %>%
+   group_by(year, sex) %>%
+   summarize(total = sum(n)) %>%
+   qplot(year, total, color = sex, data = .,
+         geom = "line") %>%
+   add(ggtitle('People named "Kim"')) %>%
+   print
```

Strategies for debugging

Debugging

What's debugging?

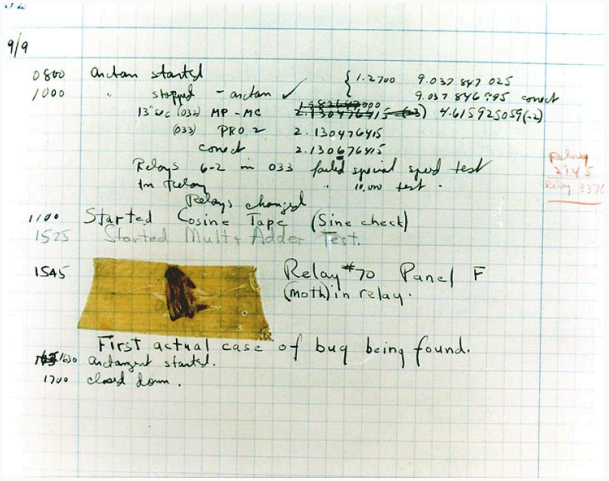
Straight from the Wikipedia

"Debugging is the process of finding and resolving bugs (defects or problems that prevent correct operation) within computer programs, software, or systems."

A famous (yet not the first) bug:

The term "bug" was used in an account by computer pioneer Grace Hopper (see on the right). While she was working on a Mark II computer at Harvard University, her associates discovered a moth stuck in a relay and thereby impeding operation, whereupon she remarked that they were "debugging" the system. This bug was carefully removed and taped to the log book (see on the right).

Above: Grace Hopper, *Below:* The bug



Why debugging matters

The Wikipedia list of software bugs with significant consequences is growing and you don't want to be on it.

NASA software engineers are famous for producing bug-free code. This was learned the hard and costly way though. Some highlights from space:

- 1962: A booster went off course during launch, resulting in the destruction of NASA Mariner 1 . This was the result of the failure of a transcriber to notice an overbar in a handwritten specification for the guidance program, resulting in an incorrect formula the FORTRAN code.
- 1999: NASA's Mars Climate Orbiter was destroyed, due to software on the ground generating commands based on parameters in pound-force (lbf) rather than newtons (N)
- 2004: NASA's Spirit rover became unresponsive on January 21, 2004, a few weeks after landing on Mars. Engineers found that too many files had accumulated in the rover's flash memory (the problem could be fixed though by deleting unnecessary files, and the Rover lived happily ever after. Until it froze to death in 2011).

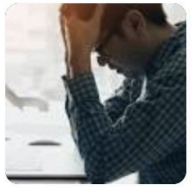
Why debugging matters (cont.)

ZDNet

Microsoft Excel blunder: Developers blamed for loss of thousands of COVID-19 test results

The error has hampered the UK's contact-tracing program at a time when the country is undergoing a second wave of coronavirus infections. By using the XLS ...

1 month ago



ABC News

Excel glitch leads to nearly 16,000 confirmed coronavirus cases going unreported in United Kingdom

For the test and trace program to work well, contacts should be notified as soon as possible. Health Secretary Matt Hancock told MPs that the problem related to ...

1 month ago

Metro

How were 16,000 Test and Trace coronavirus cases lost on Excel?

The cases were lost due to a technical error on a Microsoft Excel spreadsheet. ... Trace 'immediately' after the issue was resolved and thanked contact tracers for ...

1 month ago

Does Contact Tracing Work? Quasi-Experimental Evidence from an Excel Error in England*

Thiemo Fetzer† Thomas Graeber‡

November 24, 2020

Abstract

Contact tracing has been a central pillar of the public health response to the COVID-19 pandemic. Yet, contact tracing measures face substantive challenges in practice and well-identified evidence about their effectiveness remains scarce. This paper exploits quasi-random variation in COVID-19 contact tracing. Between September 25 and October 2, 2020, a total of 15,841 COVID-19 cases in England (around 15 to 20% of all cases) were not immediately referred to the contact tracing system due to a data processing error. Case information was truncated from an Excel spreadsheet after the row limit had been reached, which was discovered on October 3. There is substantial variation in the degree to which different parts of England areas were exposed – by chance – to delayed referrals of COVID-19 cases to to the contact tracing system. We show that more affected areas subsequently experienced a drastic rise in new COVID-19 infections and deaths alongside an increase in the positivity rate and the number of test performed, as well as a decline in the performance of the contact tracing system. Conservative estimates suggest that the failure of timely contact tracing due to the data glitch is associated with more than 125,000 additional infections and over 1,500 additional COVID-19-related deaths. Our findings provide strong quasi-experimental evidence for the effectiveness of contact tracing.

Keywords: Health, Coronavirus
JEL Classification: I31, Z18

Why debugging matters (cont.)

Technology

Facebook made big mistake in data it provided to researchers, undermining academic work

Company accidentally left out half of all of its U.S. users in providing data to a research consortium

The error resulted from Facebook accidentally excluding data from U.S. users who had no detectable political leanings — a group that amounted to roughly half of all of Facebook's users in the United States. Data from users in other countries was not affected.

"It's data. Of course, there are errors," said Gary King, a Harvard professor who co-chairs Social Science One. "This, of course, was a big error."

King, director of the university's Institute for Quantitative Social Science, said dozens of papers from researchers affiliated with Social Science One had relied on the data since Facebook shared the flawed set in February 2020, but he said the impact could be determined only after Facebook provided corrected data that could be reanalyzed. He said some of the errors may cause little or no problems, but others could be serious.

Social Science One shared the flawed data with at least 110 researchers, King said.

An Italian researcher, Fabio Giglietto, discovered data anomalies last month and brought them to Facebook's attention. The company contacted researchers in recent days with news that they had failed to include roughly half of its U.S. users — a group that likely is less politically polarized than Facebook's overall user base. The New York Times first reported Facebook's error.

`Source` Washington Post

`Source` Solomon Messing / Twitter

A general strategy for debugging

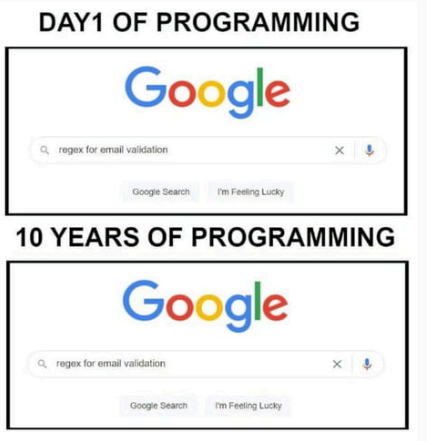
1. Google
2. Reset
3. Debug
4. Deter

Google

According to this analysis, the most common error types in R are:[1]

1. `Could not find function` errors, usually caused by typos or not loading a required package.
2. `Error in if` errors, caused by non-logical data or missing values passed to R's `if` conditional statement.
3. `Error in eval` errors, caused by references to objects that don't exist.
4. `Cannot open` errors, caused by attempts to read a file that doesn't exist or can't be accessed.
5. `no applicable method` errors, caused by using an object-oriented function on a data type it doesn't support.
6. `subscript out of bounds` errors, caused by trying to access an element or dimension that doesn't exist
7. Errors caused by failing to install/compile/load a package.

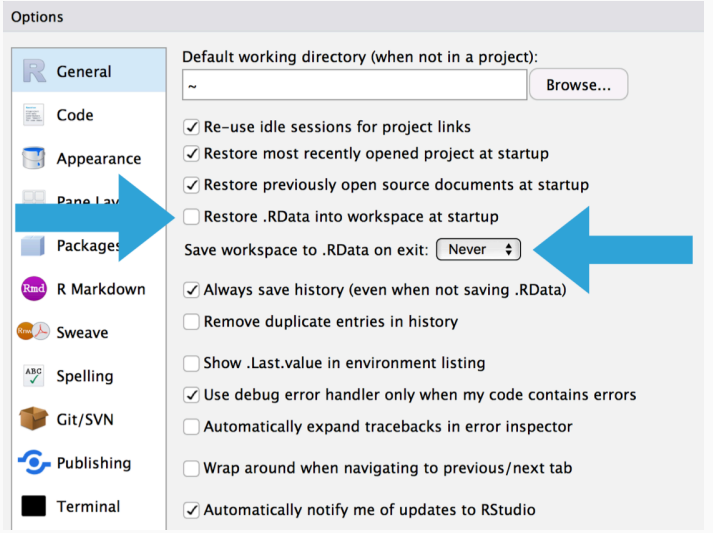
Whenever you see an error message, start by googling it. Improve your chances of a good match by removing any variable names or values that are specific to your problem. Also, look for Stack Overflow posts and list of answers.

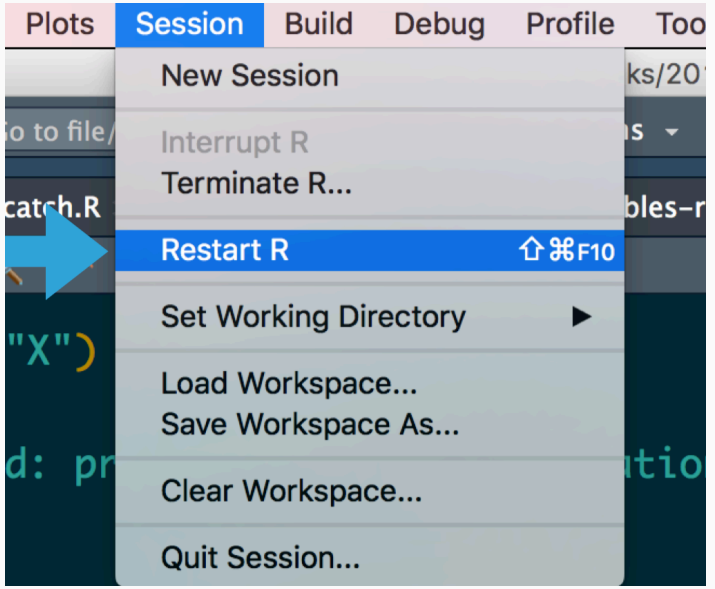


[1]Do you get an error message you don't understand? That's good news actually, because the really nasty bugs come without errors.

Reset

- If at first you don't succeed, try exactly the same thing again.
- Have you tried turning it off and on again?
- Do you use `rm(list = ls())`? Don't. Packages remain loaded, options and environment variables set, ... all possible sources of error!
- A fresh start clears the workspace, resets options, environment variables, and the path.
- While we're at it, check out James Wade's advice "How I set up RStudio for Efficient Coding" (YouTube).





Debug

Make the error repeatable.

- Execute the code many times as you consider and reject hypotheses. To make that iteration as quick possible, it's worth some upfront investment to make the problem both easy and fast to reproduce.
- Work with reproducible and minimal examples by removing innocuous code and simplifying data.
- Consider automated testing. Add some nearby tests to ensure that existing good behaviour is preserved.

Track the error down.

- Execute code step by step and inspect intermediate outputs.
- Adopt the scientific method: Generate hypotheses, design experiments to test them, and record your results.

Once found, fix the error and test it.

- Ensure you haven't introduced any new bugs in the process.
- Make sure to carefully record the correct output, and check against the inputs that previously failed.
- Reset and run again to make sure everything still works.

Deter

Defensive programming

- **Pay attention.** Do results make sense? Do they look different from previous results? Why?
- **Know what you're doing**, and what you're expecting.
  - Avoid functions that return different types of output depending on their input, e.g., `[]` and `sapply()`.
  - Be strict about what you accept (e.g., only scalars).
  - Avoid functions that use non-standard evaluation (e.g., `with()`)
- **Fail fast**.
  - As soon as something wrong is discovered, signal an error.
  - Add tests (e.g., with the `testthat` package).
  - Practice good condition/exception handling, e.g., with `try()` and `tryCatch()`.
  - Write error messages for humans.

Transparency

- Collaborate! Pair programming is an established software development technique that increases code robustness. It also works from remote.
- Be transparent! Let others access your code and comment on it.

Debugging R: What you get

```
Error : .onLoad failed in loadNamespace() for 'rJava', details:
call: dyn.load(file, DLLpath = DLLpath, ...)
error: unable to load shared object '/Users/janedoe/Library/R/3.6/library/rJava/libs/rJava.so':
libjvm.so: cannot open shared object file: No such file or directory
Error: loading failed
Execution halted
ERROR: loading failed
* removing '/Users/janedoe/Library/R/3.6/library/rJava/'
Warning in install.packages :
installation of package 'rJava' had non-zero exit status
```

`Credit` Jenny Bryan

Debugging R: What you see

```
Error : blah failed blah blah() blah 'blah', blah:
call: blah.blah(blah, blah = blah, ...)
error: unable to blah blah blah '/blah/blah/blah/blah/blah/blah/blah/blah/blah.so':
blah.so: cannot open blah blah blah: No blah blah blah blah
Error: blah failed
blah blah
ERROR: blah failed
* removing '/blah/blah/blah/blah/blah/blah/blah/'
Warning in blah.blah :
blah of blah 'blah' blah blah-blah blah blah
```

`Credit` Jenny Bryan

Strategies to debug your R code

Sometimes the mistake in your code is hard to diagnose, and googling doesn't help. Here are a couple of strategies to debug your code:

- Use `traceback()` to determine where a given error is occurring.
- Output diagnostic information in code with `print()`, `cat()` or `message()` statements.
- Use `browser()` to open an interactive debugger before the error
- Use `debug()` to automatically open a debugger at the start of a function call.
- Use `trace()` to make temporary code modifications inside a function that you don't have easy access to.

Locating errors with traceback()

Motivation and usage

- When an error occurs with an unidentifiable error message or an error message that you are in principle familiar with but cannot locate its sources, the `traceback()` function comes in handy.
- The `traceback()` function prints the sequence of calls that led to an uncaught error.
- The `traceback()` output reads from bottom to top.
- Note that errors caught via `try()` or `tryCatch()` do not generate a traceback!
- If you're calling code that you `source()`d into R, the traceback will also display the location of the function, in the form `filename.r#linenumber`.

Example

In the call sequence below, the execution of `g()` triggers an error:

```
R> f <- function(x) x + 1
R> g <- function(x) f(x)
R> g("a")
```

```
#> Error in x + 1 : non-numeric argument to binary o
```

Doing the traceback reveals that the function call f(x) is what lead to the error:

```
R> traceback()
```

```
#> 2: f(x) at #1
#> 1: g("a")
```

Interactive debugging with browser()

Motivation and usage

- Sometimes, you need more information than the precise location of an error in a function to fix it.
- The interactive debugger lets you pause the run of a function and interactively explore its state.
- Two options to enter the interactive debugger:
  1. Through RStudio's "Rerun with Debug" tool, shown to the right of an error message.
  2. You can insert a call to `browser()` into the function at the stage where you want to pause, and re-run the function.
- In either case, you'll end up in an interactive environment inside the function where you can run arbitrary R code to explore the current state. You'll know when you're in the interactive debugger because you get a special prompt, `Browse[1]>`.

Example

```
R> h <- function(x) x + 3
R> g <- function(b) {
+   browser()
+   h(b)
+ }
R> g(10)
```

Some useful things to do are:

1. Use `ls()` to determine what objects are available in the current environment.
2. Use `str()`, `print()` etc. to examine the objects.
3. Use `n` to evaluate the next statement.
4. Use `s`: like `n` but also step into function calls.
5. Use `where` to print a stack trace (→ traceback).
6. Use `c` to exit debugger and continue execution.
7. Use `Q` to exit debugger and return to the R prompt.

Debugging other peoples' code

Motivation

- Sometimes the error is outside your code in a package you're using, you might still want to be able to debug.
- Two options:
  1. Get a local version of the package code and debug as if it were your own.
  2. Use functions which which allow you to start a browser in existing functions, including `recover()` and `debug()`.

Debugging other peoples' code (cont.)

Motivation

- `recover()` serves as an alternative error handler which you activate by calling `options(error = recover)`.
- You can then select from a list of current calls to browse.
- `options(error = NULL)` turns off this debugging mode again.
- A simpler alternative is `options(error = browser)`, but this only allows you to browse the call where the error occurred.

Example

- Activate debugging mode; then execute (flawed) function:

```
R> options(error = recover)
R> lm(mpg ~ wt, data = "mtcars")
```

```
Error in model.frame.default(formula = mpg ~ wt, data = "mtcars", drop
 'data' must be a data.frame, environment, or list

Enter a frame number, or 0 to exit

1: lm(mpg ~ wt, data = "mtcars")
2: eval(mf, parent.frame())
3: eval(mf, parent.frame())

Selection:
```

- Deactivate debugging mode:

```
R> options(error = NULL)
```

Debugging other peoples' code (cont.)

Motivation

- `debug()` activates the debugger on any function, including those in packages (see on the right). `undebug()` deactivates the debugger again.
- Some functions in another package are easier to find than others. There are
  - *exported* functions which are available outside of a package and
  - *internal* functions which are only available within a package.
- To find (and debug) exported functions, use the `::` syntax, as in `ggplot2::ggplot`.
- To find un-exported functions, use the `:::` syntax, as in `ggplot2:::check_required_aesthetics`.

Example

- Activate debugging mode for `lm()` function; then execute function:

```
R> debug(stats::lm)
R> lm(mpg ~ weight, data = "mtcars")
```

- Interactive debugging mode for `lm()` is entered; use the common `browser()` functionality to navigate:

```
debugging in: lm(mpg ~ weight, data = mtcars)
debug: {
    ret.x <- x
    ...
Browse[2]>
```

- Deactivate debugging mode:

```
R> undebug(stats::lm)
```

Debug Mode

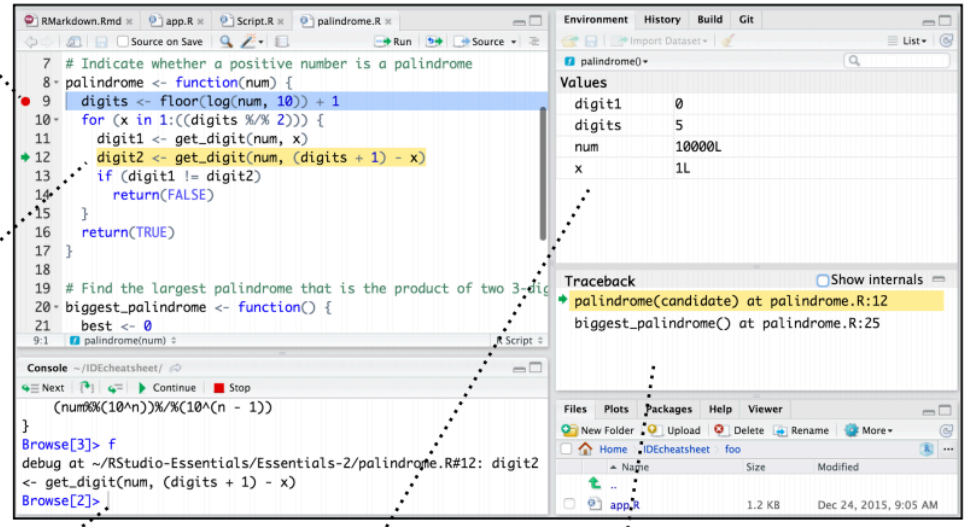
Open with **debug(), browser(),** or a breakpoint. RStudio will open the debugger mode when it encounters a breakpoint while executing code.

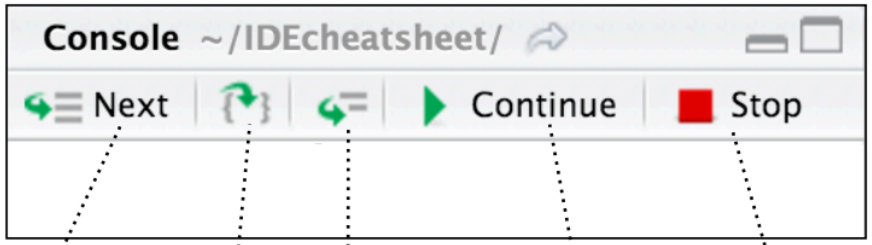
Launch debugger mode from origin of error

Open traceback to examine the functions that R called before the error occurred

Click next to line number to add/remove a breakpoint.

Highlighted line shows where execution has paused





Run commands in environment where execution has paused

Examine variables in executing environment

Select function in traceback to debug

Step through code one line at a time

Step into and out of functions to run

Resume execution

Quit debug mode

More on debugging R

Further reading

- 12-minute video on debugging in R
- Jenny Bryan's talk on debugging at rstudio::conf 2020
- Jenny Bryan and Jim Hester's "What They Forgot to Teach You About R", Chapter 11: Debugging R code
- Jonathan McPherson's Debugging with RStudio

Next steps

Assignment

Assignment 1 is online! You have a bit more than a week to work on it.

Next lecture

Programming II Buckle up and bring coffee, because it'll get both exciting and tedious at the same time. We're going to iterate, automate, and schedule.